

Manual Técnico e de Administração do Sistema Hórus

Versão do Documento: 3.0

Data de Atualização: Dezembro de 2025

Tecnologia Principal: Python (Flask) + PostgreSQL

Esta documentação foi elaborada para fornecer uma visão técnica, funcional e operacional completa do sistema Hórus. O objetivo é capacitar administradores de sistemas, mantenedores e analistas a compreender, operar, manter, monitorar e solucionar problemas na plataforma, mesmo sem conhecimento prévio do código-fonte. Ela segue as melhores práticas de engenharia de software, focando na clareza, estrutura modular e didática.

O documento é uma fusão de versões anteriores, eliminando duplicações e integrando atualizações sobre agendamento automático, integração RT e análise de senhas, para formar um guia unificado e exaustivo. Inclui detalhes expandidos sobre as funcionalidades dos scripts principais: `statistics_dashboard.py` (geração de estatísticas e gráficos), `app.py` (aplicação web principal), `credenciais_processor.py` (processamento e validação de credenciais) e `password_analyzer.py` (análise avançada de senhas).

1. Introdução e Visão Geral do Sistema

O Hórus é uma plataforma de Threat Intelligence especializada na detecção, processamento, armazenamento, análise e gestão de credenciais vazadas. O sistema atua como um centralizador que ingere dados de múltiplas fontes (e-mails de abuse, APIs de inteligência, uploads manuais e integrações externas), normaliza essas informações, valida sua relevância (com foco em domínios militares/governamentais) e alerta a equipe de segurança através de tíquetes no Request Tracker (RT). Além disso, oferece análise de força de senhas, deduplicação de registros e visualizações estatísticas para inteligência sobre segurança.

1.1 Funcionalidades Principais

- **Ingestão de Dados:** Upload de arquivos (.txt, .xls, .xlsx, .ods), inserção manual e sincronização automática via API BTTNG. Suporte a múltiplos arquivos simultâneos com verificação de duplicatas via hash MD5.
- **Processamento Inteligente:** Validação rigorosa de CPFs, e-mails e domínios (ex: .mil.br, .intraer, decea.gov.br). Uso de heurísticas, Machine Learning (ML) e regex para extração de credenciais em arquivos desformatados. O script `credenciais_processor.py` é o núcleo dessa funcionalidade, processando linhas de arquivos, validando formatos e inserindo dados no banco com deduplicação automática.
- **Deduplicação:** Separação entre registros únicos e histórico bruto de vazamentos, utilizando hashes SHA1 para comparação.
- **Análise de Senhas:** Verificação de força, complexidade e cruzamento com bases conhecidas (ex: Have I Been Pwned - HIBP). O script `password_analyzer.py` fornece análises detalhadas, incluindo distribuição de comprimento, complexidade, proporção de força (Fraca/Média/Forte) e tendências temporais. Gera relatórios executivos e gráficos em base64 para integração em dashboards.
- **Dashboard e Relatórios:** Visualização estatística, gráficos e exportação de dados (CSV). O script `statistics_dashboard.py` gera estatísticas como totais de credenciais, senhas comuns, sites com vazamentos e gráficos (ex: top sites, proporção únicos vs. duplicados). Suporte a exportação CSV de relatórios.
- **Auditoria:** Log completo de ações dos usuários no sistema, incluindo acessos, sincronizações e rejeições.
- **Automação e Alertas:** Integração com RT para abertura automática de tíquetes, com anexos de relatórios mascarados.
- **Integração BTTNG:** Sincronização automática com a API de inteligência de ameaças da Apura.
- **Interface Web:** Gerenciada pelo `app.py`, oferece rotas para login, upload, busca de usuários, relatórios, estatísticas e análise de senhas. Inclui suporte a sessões seguras, autenticação com hash de senhas e proteção contra ataques comuns.

1.2 Arquitetura Lógica

O sistema segue o padrão de arquitetura MVC (Model-View-Controller), utilizando Flask no backend e PostgreSQL como banco de dados.

O fluxo de dados opera em três estágios principais:

1. **Coleta (Ingestão):** E-mail IMAP, API BTTNG e Upload Web.
2. **Processamento (ETL):** Normalização, ML para extração de texto, validação e deduplicação.
3. **Ação (Saída):** Dashboard Web, relatórios CSV e abertura de tíquetes RT.

2. Arquitetura do Sistema

O sistema opera em uma arquitetura web clássica (MVC), com ênfase em modularidade para facilitar a manutenção.

2.1 Componentes Tecnológicos

- **Backend:** Python 3 com framework Flask.
- **Banco de Dados:** PostgreSQL (drivers psycopg2, com suporte a SSL em produção).
- **Interface:** HTML/CSS (templates Jinja2).
- **Bibliotecas de Análise:** Pandas, Matplotlib, Seaborn (para gráficos estáticos), NetworkX (para ML opcional).
- **Outras Bibliotecas:** Werkzeug (segurança), SQLAlchemy (pool de conexões).
- **Ambiente de Execução:** Suporte a containers (ex: Docker) e agendamento via CRON no Linux.

2.2 Estrutura de Arquivos

Para o mantenedor, é crucial entender onde cada lógica reside. Abaixo, uma tabela resumindo os principais arquivos, com detalhes expandidos sobre suas funcionalidades principais:

Arquivo	Função Principal
app.py	Controlador Principal (Flask). Gerencia rotas web, autenticação (com hash de senhas via Werkzeug), upload de múltiplos arquivos (com verificação de extensões e hash MD5 para deduplicação), orquestra chamadas para processamento e sincronização BTTNG. Inclui decoradores para login e admin required, logs de auditoria, exportação CSV de relatórios e integração com scheduler BTTNG em thread background. Suporte a SSL/TLS com ciphersuites seguras e headers de segurança. Rotas principais: /upload (processa arquivos), /search (busca usuários), /relatorio (gera relatórios), /estatisticas (dashboard), /analise_senhas (análise de senhas), /manage_users (gerenciamento de usuários, restrito a admins).
credenciais_processor.py	Motor de Processamento. Contém regex, validações de CPF/Domínio e lógica de inserção no banco. Processa arquivos (.txt, .xls, .xlsx), valida linhas (identificando usuário, senha e site com prioridade para CPF/email), gera hashes SHA1, insere em batch no PostgreSQL com ON CONFLICT para deduplicação. Gera arquivos de processados/rejeitados, HTML para relatórios e dashboards. Funções chave: process_line (valida e parseia linhas), inserir_no_banco (insere em credenciais e credenciais_geral), buscar_usuario (busca parcial), processar_lista_externa (integração com Zimbra). Suporte a Excel via Pandas.
config.py	Configuração. Centraliza conexões de banco, chaves de API e variáveis de ambiente. Distingue entre dev/produção.
password_analyzer.py	Motor Analítico. Gera estatísticas de complexidade, gráficos de senhas e verificação HIBP. Calcula complexidade (baseado em comprimento, maiúsculas, números, especiais), analisa vazadas comparando hashes SHA1 com arquivo HIBP local, gera gráficos (distribuição comprimento/complexidade, proporção força, sites com senhas fracas). Funções chave: analisar_senhas (análise geral), analisar_senhas_vazadas (cruzamento HIBP), exportar_relatorio_executivo (relatório métricas), analisar_tendencias (tendências temporais), validar_senha (análise individual). Usa SQLAlchemy para queries PostgreSQL.

statistics_dashboard.py	Dashboard. Gera visualizações da página inicial de estatísticas. Calcula totais (credenciais únicas/gerais, sites/usuários únicos), senhas/sites/usuários comuns, distribuição mensal. Gera gráficos em base64 (top sites, proporção únicos/duplicados). Funções chave: gerar_estatisticas_completas (compila tudo), _gerar_graficos (usa Matplotlib/Seaborn). Integra com SQLAlchemy para queries.
abuse.py	Coleta via E-mail. Conecta ao IMAP e baixa relatórios de abuse.
zimbra_parser.py	Parser Inteligente. Processa arquivos de Zimbra/Abuse com ML para extração.
bttng_integration.py	Integração BTTNG. Sincroniza dados da API Apura.
create_rt_ticket_abuse.py	Alerta de Abuse. Cria tíquetes RT para credenciais de e-mail/uploads.
create_rt_ticket_daily.py	Alerta Diário BTTNG. Cria tíquetes RT para dados da API.
daily_horus_exec.sh	Orquestrador de Abuse. Script Shell para execução sequencial diária.
.env	Segredos. Armazena senhas de banco, chaves de API (BTTNG) e credenciais de e-mail. Não versionar.

3. Configuração e Instalação

3.1 Variáveis de Ambiente (.env)

O sistema depende do arquivo .env na raiz. Use formato Ini ou TOML. Exemplo de configuração vital:

```
text

BTTNG_ENDPOINT=https://bttng.apura.com.br/
BTTNG_API_KEY=XXXXXXXXXXXXXXXXXXXXXXXXXXXX
EMAIL_PASS=...
DB_HOST=localhost
DB_USER=horus_user
DB_PASSWORD=xxxxxx
FLASK_ENV=production
RT_TOKEN=...
```

- **Nota:** Todas as credenciais sensíveis (banco, API, e-mail) devem ser gerenciadas aqui. Nunca edite diretamente no código Python.

3.2 Banco de Dados

O sistema utiliza PostgreSQL com dois contextos configurados em config.py:

1. **Auth (enter):** Gerencia usuários do Hórus (tabela creds), permissões e logs de acesso.
2. **Repo (repositorio):** Armazena dados processados (tabelas credenciais, credenciais_geral).

Em produção, exige conexão SSL. Verifique certificados em /opt/certificates/. Use pool de conexões (SQLAlchemy) para performance; ajuste pool_size se necessário.

4. Módulos de Ingestão de Dados

O Hórus possui três vetores principais de entrada. Entender cada um é crucial para diagnosticar falhas na coleta.

4.1 Coleta via E-mail (Abuse/Zimbra)

Script Responsável: abuse.py

Conecta-se à caixa postal abuse@fab.mil.br para baixar relatórios de vazamentos.

- **Protocolo:** IMAP (Porta 143).
- **Pastas Monitoradas:** "HORUS" e "Junk".
- **Funcionamento:**
 1. Verifica e-mails não lidos.
 2. Extrai corpo (HTML/Texto) e salva como .txt.
 3. Baixa anexos; converte planilhas (.ods, .xls, .xlsx) para .csv.
 4. Salva arquivos em /Dados/zimbra.
- **Nota:** Ignora erros de certificado SSL (ssl.CERT_NONE) para compatibilidade com legados.

4.2 Integração BTTNG (API Externa)

Script Responsável: bttng_integration.py

Conecta-se à API da Apura para buscar vazamentos das últimas 24 horas.

- **Configuração:** Chaves e endpoints no .env.
- **Filtros:** Busca registros novos (days=1) para evitar reproprocessamento.
- **Auditoria:** Registra na tabela audit_log o total de credenciais encontradas, validadas e rejeitadas.

4.3 Upload Manual

Via Interface Web: app.py

Permite upload de .txt ou listas manuais pelo navegador. Verifica hash MD5 para evitar duplicatas. Suporte a múltiplos arquivos, com resumo de resultados (novos, duplicados) exibido em template dedicado.

5. Motor de Processamento e Extração

Uma vez ingeridos, os dados são processados e padronizados.

5.1 O Parser Inteligente (Zimbra Parser)

Script Responsável: zimbra_parser.py

Processa arquivos em /Dados/zimbra.

- **Tecnologia:** Classe HorusCredentialExtractorML com heurísticas e scoring para identificar usuário, senha e URL.
- **Fluxo:**
 1. Lê arquivos de /Dados/zimbra.
 2. Extrai credenciais válidas.
 3. Insere no banco.
 4. Move para /Dados/zimbra_processed com timestamp.
- **Gatilho:** Aciona create_rt_ticket_abuse.py se novas credenciais forem encontradas.

5.2 Regras de Validação

Definidas em credenciais_processor.py e zimbra_parser.py:

- **CPF:** Validação matemática de dígitos verificadores.
- **Domínios:** Allowlist rígida (ex: fab.mil.br, intraer, decea.gov.br). Edite parse_site para adicionar.
- **Blacklist:** Ignora linhas com "infected", "desktop", "unknown" para evitar lixo.
- **Tratamento de Arquivos:** Suporta texto (user:pass:url) e Excel; "adivinha" campos baseado no formato.

5.3 Analisador de Senhas

Script Responsável: password_analyzer.py

- **Classificação:** Fraca, Média ou Forte baseada em comprimento e variedade (maiúsculas, números, especiais).
- **HIBP:** Verifica contra arquivo local /Dados/HIBP/100k_sha1.txt. Atualize o caminho se necessário.
- **Funcionalidades Expandidas:** Análise de tendências (média de comprimento ao longo de dias), validação individual de senhas, geração de relatórios executivos com métricas (tamanho médio, complexidade média, sites críticos).

6. Automação e Alertas (Integração RT)

O Hórus automatiza tíquetes no RT para notificação.

6.1 Alerta de Abuse

Script Responsável: create_rt_ticket_abuse.py

- **Gatilho:** Final do zimbra_parser.py.
- **Escopo:** Credenciais de e-mails/uploads (source_btng = Falso).
- **Saída:** Arquivo de texto com senhas mascaradas anexado ao tíquete.

6.2 Alerta Diário BTTNG

Script Responsável: create_rt_ticket_daily.py

- **Gatilho:** Agendado via CRON.
- **Escopo:** Credenciais da API BTTNG (source_btng = 't') da data atual.
- **Segurança:** Usa certificado cliente (rt.pem) para autenticação curl.

7. Agendamento e Execução Automática (Cron Jobs)

O funcionamento contínuo depende de tarefas agendadas no Linux (via crontab do usuário spf).

7.1 Orquestrador de Abuse

Script: daily_horus_exec.sh (/home/spf/GIT/Horus/abuse/daily_horus_exec.sh)

• **Função:**

1. Define ambiente e logs.
2. Executa abuse.py.
3. Se sucesso, executa zimbra_parser.py.

• **Log:** /var/log/horus_abuse_daily.log.

7.2 Rotina de Sincronização BTTNG

Dividida em:

1. **Sincronização (13:00):** bttng_integration.py baixa dados.
2. **Relatório (14:00):** create_rt_ticket_daily.py compila e abre tíquete.

7.3 Tabela de Agendamento (Crontab)

Horário	Script / Comando	Descrição	Log de Saída
11:00	daily_horus_exec.sh	Fluxo Abuse: Coleta, processa e abre tíquete RT se houver novidades.	/var/log/horus_abuse_daily.log
13:00	bttng_integration.py	Sync BTTNG: Baixa credenciais da API.	/home/spf/GIT/Horus/horus_bttng_sync.log
14:00	create_rt_ticket_daily.py	Tíquete BTTNG: Gera relatório dos dados baixados.	/home/spf/GIT/Horus/rt/log_rt.txt

Nota: Mantenha o relatório após a sincronização para evitar incompletos.

Nota Final

Este sistema lida com dados sensíveis. Qualquer manutenção deve preservar mecanismos de segurança, hash de senhas e conformidade com a LGPD e normativos internos. Em caso de dúvidas, consulte o código-fonte modular para extensões.